So sánh mô hình học máy và Transformer trong phân loại đa nhãn chú thích mã nguồn: Nghiên cứu thăm dò trên NLBSE'23

Tóm tắt

Một câu chú thích lớp có thể đồng thời truyền đạt nhiều loại thông tin, vì vậy việc tự động xác định chức năng của câu là bài toán phân loại đa nhãn. Nghiên cứu so sánh năm phương pháp học máy truyền thống và mười bảy cấu hình RoBERTa/CodeBERT trên 6.738 câu NLBSE'23 thuộc Python, Java và Pharo. Kết quả thực nghiệm có sẵn được tổng hợp theo các phần chia chung, có giữ nhóm văn bản trùng: sàng lọc bằng đánh giá chéo năm fold, sau đó đánh giá trên phần chia huấn luyện, xác thực và kiểm tra xấp xỉ 80/10/10. CodeBERT kết hợp R-Drop có micro-F1 trung bình cao nhất trong giai đoạn sàng lọc trên Python và Java, lần lượt đạt 64,79% và 62,25%; RoBERTa với focal loss dẫn đầu Pharo ở mức 71,52%. Chênh lệch so với RoBERTa dùng binary cross-entropy lần lượt là 3,10; 2,57 và 1,07 điểm phần trăm. Học máy truyền thống vẫn cạnh tranh trên Java, trong khi mức cải thiện của Transformer lớn hơn trên Pharo. Phân tích từng nhãn cho thấy cải thiện tổng hợp có thể đi kèm suy giảm ở nhãn ít mẫu. Do sàng lọc đã dùng toàn quần thể và mô hình nơ-ron dùng cùng fold xác thực để chọn checkpoint và chấm điểm, kết quả kiểm tra cuối chỉ có tính thăm dò. Đóng góp của bài báo là so sánh có đối soát các thực nghiệm đã hoàn thành và giới hạn đánh giá, không đề xuất kiến trúc mới hoặc khẳng định vượt baseline chính thức của cuộc thi.

Từ khóa: chú thích mã nguồn; phân loại đa nhãn; TF-IDF; RoBERTa; CodeBERT; đánh giá chéo.

1 Giới thiệu

Chú thích lớp cung cấp thông tin về mục đích, cách sử dụng, trách nhiệm và chi tiết cài đặt. Một câu có thể đồng thời thể hiện nhiều chức năng này. Việc ép mỗi câu vào đúng một lớp sẽ làm mất thông tin chú giải và không phản ánh đầy đủ các nhu cầu khai thác tài liệu. Nghiên cứu trước đã khảo sát hệ thống loại thông tin chú thích theo ngôn ngữ và khả năng phân loại tự động trên Python, Java và Smalltalk [2]; cuộc thi NLBSE'23 sau đó cung cấp tài nguyên chung cho bài toán này [1, 3].

Hai hướng mô hình hóa cần được xem xét song song. Đặc trưng từ vựng thưa kết hợp bộ phân loại truyền thống tạo các mốc so sánh có chi phí huấn luyện thấp, đồng thời phản ánh giá trị của mẫu từ và ký tự. Transformer tiền huấn luyện có khả năng biểu diễn ngữ cảnh rộng hơn, nhưng kết quả còn phụ thuộc hàm mất mát, cách mô hình hóa nhãn và quy tắc chọn checkpoint. Vì vậy, cần thống nhất dữ liệu và quy trình đánh giá trước khi diễn giải khác biệt giữa các kiến trúc.

Nghiên cứu đặt ra ba câu hỏi: **RQ1**, các phương pháp truyền thống so với RoBERTa tham chiếu như thế nào? **RQ2**, cấu hình nơ-ron nào dẫn đầu theo từng ngôn ngữ? **RQ3**, cải thiện tổng hợp có đồng thời cải thiện từng nhãn hay không? Bài báo tổng hợp 66 lượt đánh giá chéo năm fold và 30 lượt đánh giá cuối của các phương pháp có khả năng học. Kết quả ML và DL được trình bày riêng trước khi đối chiếu trên cùng quy trình. Không thực hiện thêm huấn luyện, ensemble hoặc tìm siêu tham số để viết bài báo này.

2 Cơ sở và phạm vi nghiên cứu

Binary relevance học một bộ phân loại cho từng nhãn; classifier chain còn sử dụng các nhãn đứng trước làm điều kiện dự đoán [4]. RoBERTa [5] cung cấp bộ mã hóa văn bản tiền huấn luyện, còn CodeBERT [6] khai thác cả ngôn ngữ tự nhiên và ngôn ngữ lập trình trong giai đoạn tiền huấn luyện. Focal loss điều chỉnh đóng góp của lỗi theo độ khó của dự đoán [7]; R-Drop khuyến khích hai lần truyền xuôi với dropout đưa ra phân phối nhất quán [8]. Các cơ chế này tạo cơ sở cho những biến thể được đánh giá, nhưng không bảo đảm một biến thể sẽ cải thiện trên bộ dữ liệu hiện tại.

Bộ 6.738 mẫu thuộc bài toán *phân loại chú thích mã nguồn*, khác với bài toán phân loại issue của cùng cuộc thi NLBSE'23. Baseline chính thức cho chú thích mã nguồn gồm các bộ phân loại Random Forest theo loại thông tin, sử dụng đặc trưng từ vựng và đặc trưng NLP [3]. RF trong nghiên cứu này dùng biểu diễn đặc trưng và phần chia khác, nên không phải bản tái lập chính xác baseline đó. RoBERTa với binary cross-entropy, ký hiệu E1, là *mô hình Transformer tham chiếu trong nghiên cứu*. Các so sánh định lượng dưới đây áp dụng cho quy trình chung của nghiên cứu, không phải bảng xếp hạng chính thức của cuộc thi.

3 Dữ liệu và quy trình thực nghiệm

3.1 Nhãn đa trị ở mức câu

Dữ liệu dạng dài được tổng hợp theo mã định danh câu, giữ toàn bộ chú giải nhị phân. Mỗi câu x_i có vector đích $\mathbf{y}_i \in \{0,1\}^L$, với $L = 5$ cho Python và $L = 7$ cho Java, Pharo. Các loại thông tin gắn với từng ngôn ngữ; mô hình được huấn luyện riêng, không gộp mười chín loại vào một bài toán dự đoán chung.

Văn bản chỉ được chuẩn hóa khoảng trắng trước khi tạo nhóm. Các câu có văn bản chuẩn hóa giống hệt nhau luôn thuộc cùng fold hoặc phần chia cuối, kể cả khi mã định danh khác nhau. Python, Java và Pharo lần lượt có 2.299; 1.668 và 1.620 nhóm văn bản phân biệt. Tần suất trùng văn bản cho thấy việc chia ngẫu nhiên từng dòng riêng lẻ có thể gây rò rỉ. Nhãn vẫn được giữ theo mã câu; chú giải khác nhau cho cùng văn bản không bị tự động quy thành lỗi gán nhãn.

Bảng 1 mô tả quy mô quần thể. Phân bố nhãn mất cân bằng: Java có 115 mẫu dương cho *Ownership* so với 912 cho *usage*; Pharo có 77 mẫu dương cho *Classreferences* so với 748 cho *Example*. Số nhãn dương trung bình gần một, trong khi mỗi câu được đánh giá trên năm hoặc bảy nhãn, nên đích âm chiếm ưu thế. Cần kết hợp độ đo tổng hợp và độ đo phản ánh khác biệt giữa các nhãn.

3.2 Đánh giá hai giai đoạn

Stage 1: sàng lọc. Toàn bộ câu tham gia năm fold có giữ nhóm. Phép gán nhóm cân bằng gần đúng số câu và số nhãn dương, không bảo đảm phân tầng hoàn hảo. Mỗi phần huấn luyện của fold học lại bộ trích xuất đặc trưng và bộ phân loại, hoặc khởi tạo mô hình nơ-ron từ trọng số tiền huấn luyện. Thống kê TF-IDF, trọng số lớp và thống kê đồ thị nhãn chỉ

Bảng 1: Quần thể dữ liệu và các phần chia giai đoạn cuối. Cardinality là số nhãn dương trung bình trên mỗi câu.

Ngôn ngữ	Số câu	Số nhãn	Huấn luyện	Xác thực	Kiểm tra	Cardinality
Python	2555	5	2044	255	256	1,121
Java	2418	7	1935	241	242	1,177
Pharo	1765	7	1413	176	176	1,158

được ước lượng từ phần huấn luyện tương ứng. Các phương pháp dùng chung fold và thứ tự nhãn trong mỗi ngôn ngữ. Trung bình và độ lệch chuẩn mẫu (ĐLC) được tính từ năm điểm theo fold; ĐLC không phải khoảng tin cậy.

Stage 2: đánh giá cuối đã thực hiện. Cùng quần thể được sắp xếp thành các phần huấn luyện, xác thực và kiểm tra xấp xỉ 80/10/10, có giữ nhóm văn bản trùng. ML chỉ fit trên train; validation phục vụ chẩn đoán. DL dùng micro-F1 xác thực để chọn checkpoint trước khi đánh giá test. Cả năm phương pháp ML và các cấu hình E1/E3/E9/E15/E17 đều có kết quả cuối trên ba ngôn ngữ. Tập cấu hình này không bao phủ đầy đủ top bốn DL theo CV: Python thiếu E13/E11, Java thiếu E12, Pharo thiếu E8/E12. Kết quả chưa chạy không được suy diễn hoặc điền điểm.

Ràng buộc diễn giải. Sàng lọc Stage 1 đã dùng mọi câu, bao gồm các câu sau đó được gán vào test của Stage 2. Vì vậy, test cuối không độc lập với việc lựa chọn phương pháp. Ngoài ra, mỗi fold DL sử dụng validation vừa để chọn checkpoint vừa để chấm điểm fold. Đây là các thực nghiệm sàng lọc đã hoàn thành, không phải CV lồng nhau hoặc đánh giá xác nhận trên tập giữ riêng từ đầu. Mô hình đại diện được chọn theo micro-F1 trung bình của Stage 1 và giữ cố định khi phân tích Stage 2.

3.3 Độ đo

Với nhãn j, ký hiệu TP_j, FP_j, FN_j là số dương tính đúng, dương tính giả và âm tính giả. Các độ đo chính gồm

$$F_{1,\text{micro}} = \frac{2\sum_j TP_j}{2\sum_j TP_j + \sum_j FP_j + \sum_j FN_j}, \quad (1)$$

$$F_{1,\text{macro}} = \frac{1}{L}\sum_{j=1}^{L} \frac{2TP_j}{2TP_j + FP_j + FN_j}, \quad (2)$$

trong đó F1 không xác định của một nhãn được gán bằng không. Hamming loss được tính bởi

$$HL = \frac{1}{nL}\sum_{i=1}^{n}\sum_{j=1}^{L} \mathbb{1}[y_{ij} \neq \hat{y}_{ij}]. \quad (3)$$

Micro-F1 là tiêu chí xếp hạng chính; macro-F1 phản ánh khác biệt giữa các nhãn, còn HL thấp biểu thị ít lỗi trên các đích nhị phân hơn. Micro average precision (AP) tổng hợp khả năng xếp hạng các cặp câu–nhãn mà không chọn ngưỡng phân loại. Điểm quyết định của SVM có thể dùng để tính AP theo thứ hạng, nhưng không phải xác suất đã hiệu chỉnh. So sánh giữa các ngôn ngữ chỉ mang tính mô tả vì không gian nhãn và phân bố mẫu khác nhau.

4 Các phương pháp được đánh giá

4.1 Học máy truyền thống

Năm phương pháp cùng dùng phép nối TF-IDF unigram/bigram từ với TF-IDF 3–5-gram ký tự trong từ. Mỗi nhánh giữ tối đa 20.000 đặc trưng, với tần suất tài liệu tối thiểu hai. TF-IDF dùng chế độ viết thường mặc định, còn phép tạo nhóm văn bản vẫn giữ phân biệt hoa/thường. Từ vựng và IDF chỉ được học trên phần huấn luyện tương ứng.

LR là logistic regression one-vs-rest, $C = 5$, trọng số lớp cân bằng và giới hạn L-BFGS 2.000 vòng. **Chain** dùng cùng bộ phân loại logistic trong classifier chain với một thứ tự nhãn ngẫu nhiên có seed; không đánh giá ensemble chain. **SVM** là Linear SVM one-vs-rest, $C = 1$ và trọng số cân bằng. **SGD** gồm các bộ phân loại logistic loss one-vs-rest, trọng số cân bằng, giới hạn 2.000 vòng và dung sai 10^{-3}. **RF** sử dụng Random Forest one-vs-rest với 300 cây và trọng số cân bằng. Đây là cấu hình cố định trong các thực nghiệm đã lưu, không phải kết quả của một đợt grid search mới. Đối chứng Dummy đa số được loại khỏi bảng so sánh của bài báo; số liệu gốc vẫn được giữ nguyên.

4.2 Học sâu

Mô hình tham chiếu ánh xạ biểu diễn token đầu tiên có 768 chiều thành L logit, sau đó áp dụng sigmoid độc lập. Hàm mục tiêu cơ sở là binary cross-entropy trung bình:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{nL}\sum_{i,j}\left[y_{ij}\log p_{ij} + (1-y_{ij})\log(1-p_{ij})\right]. \quad (4)$$

Nhóm RoBERTa gồm E1 (BCE), E2 (BCE có trọng số lớp dương), E3 (focal), E4 (hoãn tối ưu ngưỡng), E5 (attention theo nhãn), E6 (attention kèm điều chuẩn tương phản), E7 (asymmetric loss), E8 (R-Drop), E9 (đồ thị nhãn), E10 (nhiễu đối kháng trên embedding) và E12 (attention kết hợp đồ thị). Nhóm CodeBERT gồm E11 (BCE), E13 (learning rate riêng cho đầu phân loại), E14 (learning rate giảm theo lớp), E15 (R-Drop), E16 (giảm learning rate kết hợp R-Drop) và E17 (focal).

E3/E17 dùng focal loss nhị phân $-(1-p_t)^{\gamma}\log p_t$, với $\gamma = 2$ và không dùng trọng số alpha; $p_t = p$ khi đích dương, bằng $1-p$ khi đích âm. R-Drop thực hiện hai lần truyền xuôi lúc huấn luyện, lấy trung bình hai loss có giám sát và cộng phạt KL đối xứng giữa các phân phối Bernoulli của từng nhãn. E15 đặt hệ số phạt bằng 1,0. Thành phần nhất quán này không dùng softmax trên các nhãn vốn không loại trừ nhau.

Cấu hình huấn luyện chung dùng batch size 16, tối đa mười epoch, đầu vào dài tối đa 128 token, AdamW với learning rate cơ sở 2×10^{-5}, weight decay 0,01, warmup 0,1, lịch giảm tuyến tính và ngưỡng cắt gradient 1,0. Patience thường bằng hai; E9 dùng patience ba, tối thiểu năm epoch và AP để phân xử khi micro-F1 xác thực bằng nhau. E13/E14/E16 dùng learning rate 10^{-4} cho đầu phân loại; các biến thể decay còn giảm learning rate theo lớp mã hóa. Seed cơ sở là 42. ML chạy trên CPU, DL trên GPU; tổng thời gian không tạo thành phép so sánh tốc độ được kiểm soát. Dự đoán dùng quy tắc mặc định đã lưu, bao gồm ngưỡng xác suất 0,5 của DL. E4 dùng ngưỡng 0,5 lúc sàng lọc và

Bảng 2: Micro-F1 Stage 1 (%, trung bình ± ĐLC mẫu trên năm fold chung). In đậm đánh dấu trung bình cao nhất trong từng nhóm ML/DL và ngôn ngữ. E1 là Transformer tham chiếu. Ký hiệu in đậm không biểu thị ý nghĩa thống kê.

Phương pháp	Python	Java	Pharo
LR	58,33 ± 1,26	**59,72 ± 5,66**	64,38 ± 2,23
Chain	56,23 ± 1,93	57,89 ± 6,75	62,09 ± 2,47
SVM	57,30 ± 0,95	58,11 ± 5,77	63,72 ± 2,10
SGD	58,11 ± 0,93	59,32 ± 5,56	**64,86 ± 1,88**
RF	**58,82 ± 1,26**	57,26 ± 4,05	64,58 ± 2,26
E1	61,69 ± 1,77	59,68 ± 6,88	70,44 ± 3,74
E2	61,61 ± 1,99	59,81 ± 5,93	69,12 ± 4,45
E3	62,19 ± 2,56	59,54 ± 5,90	**71,52 ± 2,33**
E4	61,69 ± 1,77	59,68 ± 6,88	70,44 ± 3,74
E5	60,80 ± 2,13	60,55 ± 4,88	69,92 ± 3,18
E6	61,58 ± 1,67	61,15 ± 6,88	70,36 ± 2,78
E7	60,81 ± 1,02	59,18 ± 5,64	67,36 ± 2,08
E8	62,71 ± 2,85	60,65 ± 5,21	71,04 ± 3,07
E9	62,11 ± 2,68	61,25 ± 5,55	71,24 ± 3,04
E10	62,60 ± 1,96	59,57 ± 4,62	69,50 ± 2,53
E11	63,97 ± 1,81	59,80 ± 3,00	68,05 ± 3,58
E12	61,32 ± 1,81	61,17 ± 5,19	70,68 ± 3,67
E13	64,55 ± 1,63	60,01 ± 2,84	68,93 ± 3,47
E14	63,07 ± 1,96	60,32 ± 2,05	67,67 ± 2,82
E15	**64,79 ± 2,37**	**62,25 ± 4,16**	69,56 ± 4,50
E16	63,76 ± 1,75	59,79 ± 4,04	67,48 ± 2,02
E17	64,52 ± 2,19	61,54 ± 2,59	69,93 ± 2,87

chưa có đánh giá cuối, nên chưa xác lập lợi ích của tối ưu ngưỡng.

5 Kết quả

5.1 Sàng lọc toàn bộ phương pháp

RQ1: ML so với E1. RF dẫn đầu nhóm truyền thống trên Python ở mức $58,82 \pm 1,26\%$, LR dẫn đầu Java với $59,72 \pm 5,66\%$, SGD dẫn đầu Pharo với $64,86 \pm 1,88\%$ (Bảng 2). E1 lần lượt đạt 61,69%; 59,68% và 70,44%. Trên Java, ML tốt nhất gần như bằng E1, chỉ chênh 0,04 điểm phần trăm. Mức cao hơn của E1 so với ML được chọn trên Python và Pharo lần lượt khoảng 2,87 và 5,58 điểm. Kết quả không ủng hộ khẳng định DL luôn vượt ML ở mọi ngôn ngữ.

RQ2: cấu hình DL dẫn đầu. E15, CodeBERT kết hợp R-Drop, đứng đầu Python ($64,79 \pm 2,37\%$) và Java ($62,25 \pm 4,16\%$). E3, RoBERTa với focal loss, đứng đầu Pharo ($71,52 \pm 2,33\%$). Chênh lệch tính từ trung bình chưa làm tròn so với E1 là 3,10; 2,57 và 1,07 điểm phần trăm. Đây là mô tả thực nghiệm: dùng chung fold cho phép đối chiếu tương ứng, nhưng năm điểm fold có phụ thuộc, một seed và cơ chế chọn checkpoint chưa đủ chứng minh ý nghĩa thống kê hoặc nguyên nhân cải thiện.

Tiền huấn luyện CodeBERT không tự giải thích đầy đủ thứ hạng: E11 thấp hơn E15 trên Python, Java, trong khi mô hình đứng đầu Pharo dùng RoBERTa. Mẫu kết quả này gợi ý cần thực nghiệm được kiểm soát về tương tác giữa backbone và hàm mục tiêu, thay vì xem lựa chọn backbone là điều kiện đủ. Bảng 2 cũng cho thấy attention hoặc đồ thị bổ sung không luôn vượt mô hình tham chiếu đơn giản hơn.

5.2 So sánh giai đoạn cuối

Micro-F1 test cao nhất quan sát được thuộc Python E15 (65,41%), Java E17 (70,34%) và Pharo E1 (73,71%). Các giá trị cực đại này chỉ mô tả test, không thay thế lựa chọn bằng CV. Hình 1 vẫn theo dõi các phương pháp đã được chốt ở Stage 1.

Trên Python, E15 cao hơn E1 4,67 điểm micro-F1 test; macro-F1 tăng từ 55,47% lên 59,65%, còn HL giảm từ 16,56% xuống 14,37%. Micro-AP cũng tăng từ 65,71% lên 75,85%, cho thấy cải thiện quan sát được không chỉ giới hạn ở nhãn dự đoán cứng tại một ngưỡng.

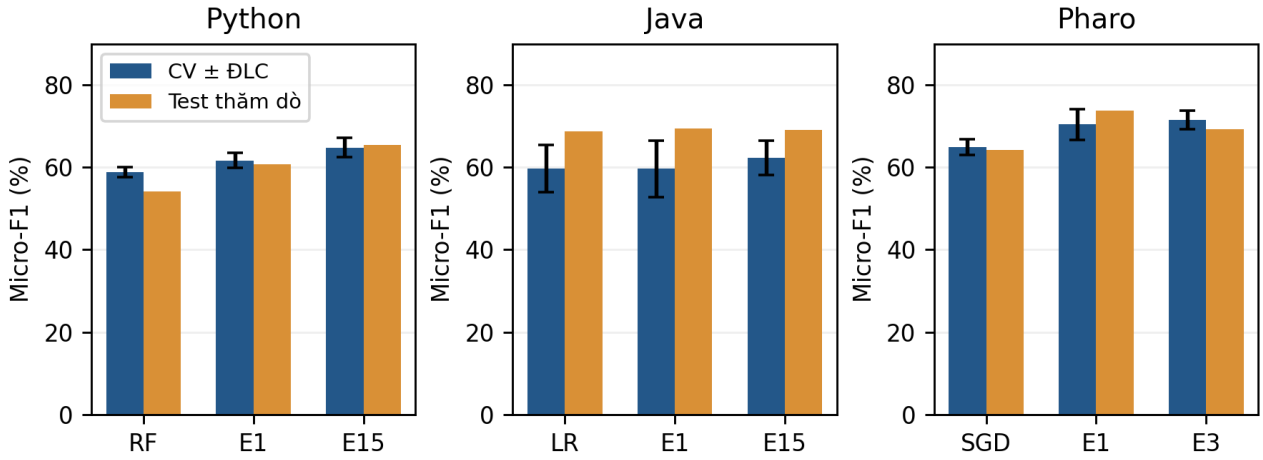
Java minh họa sự cần thiết của nhiều độ đo. Chain đạt micro-F1 69,64% và macro-F1 74,70%, so với 69,47% và 69,46% của E1. Tuy nhiên, HL của Chain cao hơn, 10,04% so với 9,45%. E17 có micro-F1 test cao nhất quan sát được, trong khi E15 vẫn là mô hình DL được chọn bằng CV. Chọn lại E17 sau khi xem test sẽ làm thay đổi quy tắc lựa chọn.

Trên Pharo, E1 vượt cả bốn cấu hình DL còn lại đã được đánh giá tại ngưỡng mặc định. E3 được ưu tiên theo CV nhưng micro-F1 test chỉ đạt 69,19%, so với 73,71% của E1. Micro-AP của hai phương pháp gần nhau hơn, lần lượt 79,56% và 79,90%. Chất lượng xếp hạng và chất lượng phân loại tại ngưỡng vì vậy có thể dẫn đến diễn giải khác nhau; các thực nghiệm hiện tại chưa cô lập nguyên nhân đảo thứ hạng.

5.3 Phân tích từng nhãn

RQ3: cải thiện tổng hợp và từng nhãn. Hình 2 cho thấy tác động không đồng nhất. Trên Python, E15 tăng F1 của *Usage* từ 72,19% lên 77,24%, *Parameters* từ 64,60% lên 71,43% và *Summary* từ 67,50% lên 71,58%. Ngược lại, *DevelopmentNotes* giảm từ 34,62% xuống 28,57%, dù điểm tổng hợp tăng. Nhãn này có 31 mẫu dương test; E15 tạo 24 âm tính giả và 11 dương tính giả. Các đếm lỗi cho thấy recall thấp ở nhãn này, nhưng chưa xác lập nguyên nhân ngữ nghĩa cụ thể.

Khác biệt về số mẫu dương cũng giới hạn diễn giải. Một vài dự đoán thay đổi có thể ảnh hưởng đáng kể F1 của nhãn hiếm, trong khi micro-F1 chịu tác động lớn hơn từ các



Hình 1: ML được chọn bằng CV, E1 và DL được chọn bằng CV theo từng ngôn ngữ. Cột CV biểu diễn trung bình và ĐLC giữa fold; cột test mô tả cùng các phương pháp cố định trên phần chia khác. Khoảng cách giữa hai cột không phải ước lượng độc lập về sai số khái quát hóa.

Bảng 3: Toàn bộ kết quả Stage 2 được đưa vào bài báo (%). Các cột là micro-F1 (μ), macro-F1 (M) và Hamming loss (HL, thấp hơn tốt hơn). Mỗi điểm dùng một phần chia cố định và một seed; test mang tính thăm dò.

	Python			Java			Pharo		
Phương pháp	μ	M	HL	μ	M	HL	μ	M	HL
LR	56,75	52,44	19,53	68,71	73,44	10,27	64,00	56,53	11,69
Chain	55,77	52,75	19,45	69,64	74,70	10,04	61,61	51,97	12,74
SVM	57,04	52,95	19,06	68,38	73,35	10,15	63,08	56,01	11,69
SGD	56,90	52,68	19,53	69,50	74,41	10,15	64,20	57,88	11,77
RF	54,21	50,19	19,53	63,60	64,95	12,16	64,47	56,07	11,36
E1	60,74	55,47	16,56	69,47	69,46	9,45	73,71	67,20	8,28
E3	59,00	54,96	17,27	68,39	71,11	10,09	69,19	60,15	9,90
E9	62,85	56,34	15,70	67,62	73,71	10,68	71,57	60,58	9,09
E15	65,41	59,65	14,37	69,14	72,84	9,33	70,50	61,94	9,17
E17	62,48	56,69	14,92	70,34	73,42	9,21	70,80	62,83	9,17

đích dương phổ biến. Macro-F1 bổ sung góc nhìn theo nhãn, nhưng không cách lấy trung bình nào bảo đảm cải thiện mọi loại thông tin. Vì vậy, chỉ báo cáo micro-F1 cao nhất sẽ che khuất các loại chú thích mà mô hình được chọn còn yếu.

6 Thảo luận và nguy cơ đối với tính hợp lệ

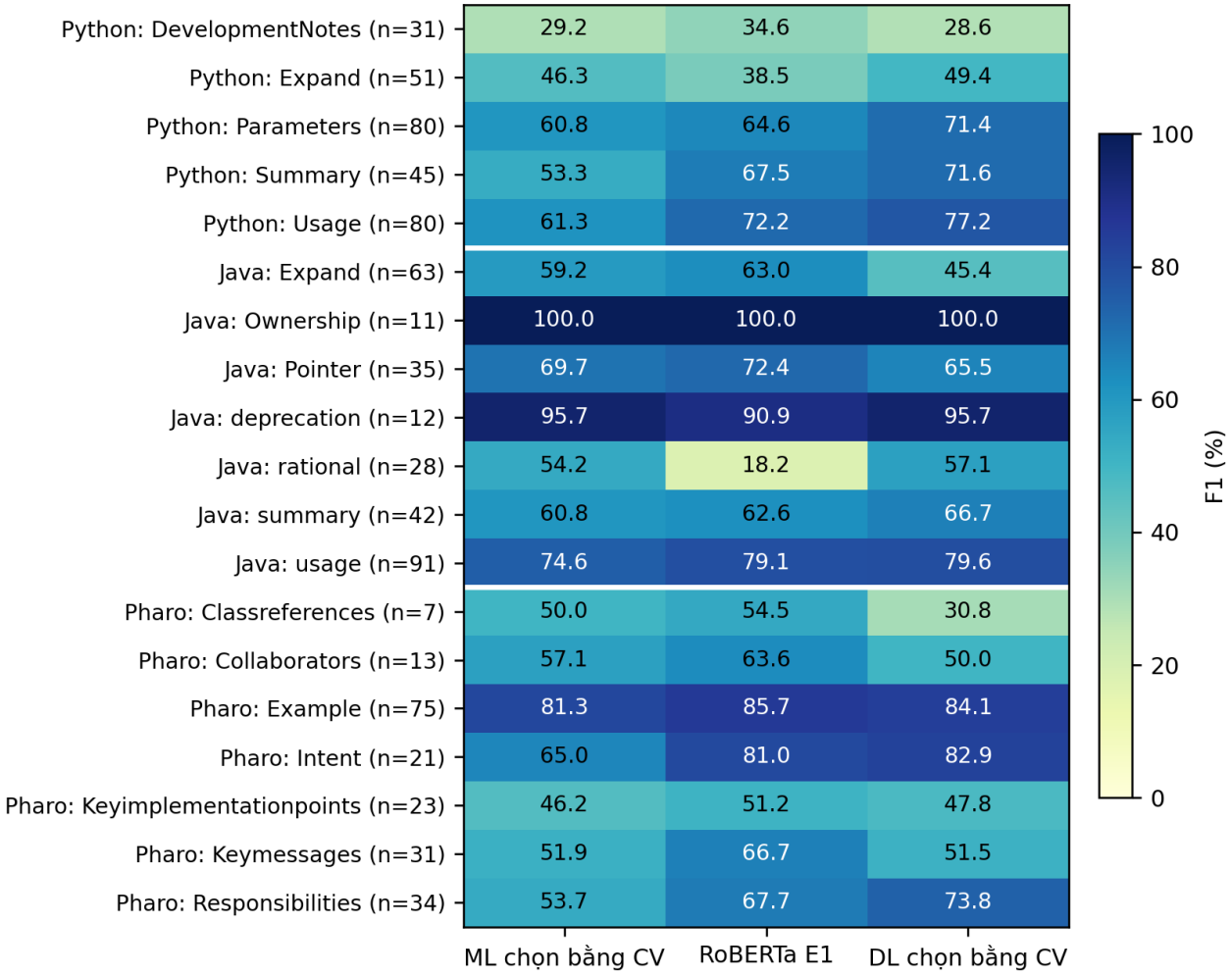
Lựa chọn phương pháp. Không có họ mô hình dẫn đầu đồng nhất. Các mô hình tuyến tính cần được giữ trong so sánh với bộ mã hóa ngữ cảnh, đặc biệt trước kết quả ML cạnh tranh trên Java. R-Drop và focal loss là ứng viên đáng khảo sát tiếp, nhưng bằng chứng hiện có phù hợp với đánh giá theo từng ngôn ngữ hơn là khuyến nghị phổ quát. Thứ hạng CV khác test thể hiện độ nhạy với phần chia, không chứng minh một phương pháp vốn có tính bất ổn.

Tính hợp lệ nội tại. Dùng toàn quần thể để sàng lọc trước khi xác định test tạo sự phụ thuộc lựa chọn. Chọn checkpoint DL trên fold được chấm điểm còn có thể làm đánh giá lạc quan hơn. Thiết kế xác nhận cần giữ riêng nhóm test trước mọi quyết định mô hình, sau đó chỉ lựa chọn trong dữ liệu phát triển, hoặc dùng CV lồng nhau có giữ nhóm. Đây là hướng đề xuất, chưa phải thực nghiệm đã hoàn thành. Bài báo cũng không tuyên bố kết quả tìm ngưỡng hoặc ensemble mới.

Tính hợp lệ của phép đo. Micro-F1, macro-F1, HL và AP phản ánh các khía cạnh khác nhau, không trực tiếp đo mức hữu ích của tài liệu, độ hiệu chỉnh xác suất hoặc năng suất lập trình viên. Phần chia theo câu mới khác phần chia theo từng loại thông tin của cuộc thi; bộ đặc trưng cũng khác baseline RF chính thức. Vì vậy, nghiên cứu không xác lập việc vượt các baseline chính thức. Các tệp curve ML có sẵn đo Hamming loss trên train theo ngân sách fit lại, không phải lịch sử tối ưu logistic loss hoặc hinge loss; chúng không được xem là đường loss tương đương với DL.

Khả năng khái quát hóa và tính hợp lệ thống kê. Giữ nhóm văn bản trùng hệt không ngăn văn bản gần trùng hoặc các lớp/dự án liên quan xuất hiện ở nhiều phần chia. Thực nghiệm chưa kiểm tra chuyển giao sang dự án tách biệt hoặc ngôn ngữ chưa thấy. Chỉ có một seed cơ sở, chưa lặp nhiều seed hoặc kiểm định ý nghĩa thống kê. ĐLC giữa fold mô tả biến động của các fold cụ thể, không bao quát mọi nguồn ngẫu nhiên huấn luyện. Riêng Linear SVM đã lưu chưa cố định random state của solver. Khác biệt CPU/GPU và chi phí chẩn đoán cũng không cho phép coi tổng thời gian là phép so sánh hiệu quả tính toán công bằng.

Khả năng đối soát bằng chứng. Toàn bộ 102 tệp metric gốc, gồm cả đối chứng Dummy không đưa vào bảng, đã được kiểm tra với hash đã lưu. Các bảng của bài báo sử dụng 96 lượt đánh giá: 66 CV và 30 đánh giá cuối. Trung bình và ĐLC mẫu được tính lại; fold chung và fingerprint quần thể được đối chiếu; bảng tổng hợp được kiểm tra với số đo gốc. Các kiểm tra hỗ trợ tính nhất quán khi tổng hợp,



Hình 2: F1 test theo nhãn (%) của ML và DL được chọn bằng CV, cùng E1. ML được chọn là RF/LR/SGD cho Python/Java/Pharo; DL là E15/E15/E3. Ký hiệu n là số mẫu dương trong test, không phải quần thể huấn luyện. Các cột được chốt trước khi phân tích các điểm này.

nhưng không loại bỏ giới hạn của thiết kế đánh giá.

7 Kết luận

Nghiên cứu so sánh năm phương pháp học máy truyền thống và mười bảy cấu hình Transformer cho phân loại đa nhãn chú thích mã nguồn NLBSE'23. CodeBERT với R-Drop dẫn đầu sàng lọc trên Python, Java; RoBERTa với focal loss dẫn đầu Pharo. ML truyền thống vẫn cạnh tranh trên Java và kết quả theo nhãn cho thấy cải thiện tổng hợp có thể che khuất suy giảm cục bộ. Các điểm test đã ghi nhận là bằng chứng thăm dò theo quy trình chung; đánh giá xác nhận cần test được dành trước lựa chọn, lặp nhiều seed và kiểm tra mức chuyển giao dự án hoặc ngôn ngữ phù hợp với ứng dụng.

Tài liệu tham khảo

[1] R. Kallis, M. Izadi, L. Pascarella, O. Chaparro, and P. Rani, "The NLBSE'23 Tool Competition," in *Proc. 2nd International Workshop on Natural Language-Based Software Engineering*, 2023. Competition description.

[2] P. Rani, S. Panichella, M. Leuenberger, A. Di Sorbo, and O. Nierstrasz, "How to identify class comment types? A multi-language approach for class comment classification," *Journal of Systems and Software*, vol. 181, art. 111047, 2021. doi:10.1016/j.jss.2021.111047.

[3] NLBSE'23 organizers, "Code comment classification: dataset and baseline classifiers," 2023. Official dataset and baseline repository.

[4] J. Read, B. Pfahringer, G. Holmes, and E. Frank, "Classifier chains for multi-label classification," *Machine Learning*, vol. 85, pp. 333–359, 2011. doi:10.1007/s10994-011-5256-5.

[5] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," arXiv:1907.11692, 2019.

[6] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of EMNLP*, pp. 1536–1547, 2020. doi:10.18653/v1/2020.findings-emnlp.139.

[7] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. ICCV*, pp. 2980–2988, 2017.

[8] X. Liang et al., "R-Drop: Regularized dropout for neural networks," *Advances in Neural Information Processing Systems*, vol. 34, pp. 10890–10905, 2021.